Étude de la robustesse des agrégateurs en apprentissage fédéré face aux attaques byzantines avec ByzFL

Arezki Célia, Abbes Samy
Parcours IAAA — 2025–2026

Résumé — L'apprentissage fédéré permet d'entraîner un modèle global à partir de données distribuées chez plusieurs clients, sans centralisation des données brutes. Ce cadre est cependant vulnérable aux *attaques byzantines non ciblées*, dont l'objectif est de dégrader ou de casser complètement le modèle global. Ce travail compare, à l'aide de la bibliothèque ByzFL, douze agrégateurs robustes face à huit attaques, sur le jeu de données MNIST en distributions IID et Non-IID. Deux expériences particulièrement éclairantes sont analysées en détail : l'attaque IPM en IID et l'attaque Gaussian en Non-IID. Nos résultats montrent qu'aucun agrégateur n'est universellement optimal et que l'hétérogénéité Non-IID amplifie considérablement la difficulté du problème.

Mots clés — apprentissage fédéré, attaques byzantines, agrégation robuste, IID, Non-IID, ByzFL, MNIST

1. INTRODUCTION

L'apprentissage fédéré (*Federated Learning*, FL) est un paradigme d'apprentissage automatique distribué proposé par McMahan *et al.* [1]. Son principe fondateur est simple, plutôt que de centraliser les données sur un serveur, on laisse chaque participant, appelé *client*, entraîner un modèle localement sur ses propres données, puis envoyer uniquement les paramètres appris (gradients ou poids) à un serveur central chargé de les agréger. Le modèle global est ainsi mis à jour sans que les données brutes ne quittent jamais les appareils des participants.

Ce paradigme est particulièrement intéressant dans des contextes où la confidentialité des données est cruciale comme la santé (dossiers médicaux), la téléphonie mobile (historique de navigation, saisie au clavier), ou la finance. Il permet de profiter de la richesse collective des données distribuées sans violer la vie privée des utilisateurs.

Cependant, cette architecture distribuée introduit une vulnérabilité fondamentale. Le serveur ne peut pas vérifier le contenu des mises à jour reçues. Un client malveillant, dit *byzantin*, peut donc envoyer des gradients arbitrairement corrompus pour dévier ou détruire l'apprentissage. Ce défi s'inscrit dans le cadre général du *problème du consensus* en systèmes distribués [2], où le serveur doit obtenir un accord correct et convergent malgré la présence de participants hostiles. Cette recherche d'un consensus robuste constitue l'objectif centrale étudiée dans ce travail.

Ce rapport présente les expériences réalisées avec la bibliothèque ByzFL [12], en comparant systématiquement douze agrégateurs robustes face à huit types d'attaques, sur le jeu de données MNIST, en distributions IID et Non-IID.

2. APPRENTISSAGE FÉDÉRÉ

2.1. Fonctionnement général

Dans un système d'apprentissage fédéré classique, l'objectif est d'optimiser un modèle d'apprentissage automatique, comme un réseau de neurones adapté à la classification d'images. Le modèle est défini par un vecteur de paramètres mathématiques appelés les poids du modèle et notés w. L'apprentissage consiste à minimiser une fonction de coût globale qui mesure l'erreur entre les prédictions du modèle et la réalité.

Le protocole se déroule en plusieurs iterations successives. À chaque iteration t, le serveur distribue les paramètres du modèle global courant w^t à l'ensemble des n clients. Chaque client i évalue l'erreur du modèle sur ses propres données locales. Il calcule ensuite un gradient g_i^t. Ce gradient correspond à la dérivée de la fonction d'erreur. Plus simplement, c'est un vecteur qui indique très précisément dans quelle direction et avec quelle force il faut modifier les poids du modèle pour réduire l'erreur locale. Le client envoie ensuite ce vecteur au serveur.

Celui-ci agrège les contributions reçues selon une règle mathématique $\mathcal{A}$ pour produire la mise à jour globale

$$w^{t+1} = w^t - \eta \cdot \mathcal{A}(g_1^t, \ldots, g_n^t)$$

où η est le taux d'apprentissage. Le modèle converge progressivement vers une solution exploitant l'ensemble des données distribuées, sans jamais avoir besoin de les centraliser.

2.2. Vulnérabilité aux clients byzantins

Parmi les n clients, f peuvent être byzantins. Ces participants hostiles sont libres d'envoyer n'importe quel vecteur, indépendamment de leurs données réelles ou même de toute donnée. L'hypothèse classique impose $f < n/2$, assurant ainsi une majorité honnête [3]. Si l'agrégateur est une simple moyenne (*Average*), même un seul client byzantin peut potentiellement envoyer un gradient de norme arbitrairement grande et dévier complètement l'apprentissage. C'est cette fragilité qui motive l'étude des agrégateurs robustes.

2.3. Deux grandes familles d'attaques

En apprentissage fédéré, les attaques byzantines se divisent en deux grandes familles. Les **attaques ciblées**, également connues sous le nom d'attaques adversarielles ou *backdoor attacks*, cherchent à faire classifier correctement l'ensemble des données normales tout en introduisant subtilement un comportement erroné sur un sous-ensemble précis de données possédant un marqueur secret. Les **attaques non ciblées** visent un objectif plus brutal en cherchant à dégrader ou casser entièrement le modèle global, sans se soucier de maintenir une apparence de normalité.

Dans ce travail, nous nous concentrons exclusivement sur les attaques non ciblées. L'ensemble des huit attaques étudiées cherchent à empêcher la convergence du modèle ou à en réduire dramatiquement les performances en cor-

rompant les mises à jour envoyées au serveur.

3. JEU DE DONNÉES ET DISTRIBUTIONS

3.1. MNIST

Le jeu de données MNIST (*Modified National Institute of Standards and Technology*) [10] est le support expérimental de toutes les expériences menées dans ce travail. Il se compose de **70 000 images en niveaux de gris** représentant des chiffres manuscrits de 0 à 9, soit **10 classes**. Chaque image est de taille **28×28 pixels**, représentée comme un vecteur de 784 valeurs entre 0 et 255.

La répartition est la suivante : **60 000 images d'entraînement** et **10 000 images de test**. Les classes sont quasiment équilibrées , environ **6 000 images par classe** dans l'ensemble d'entraînement, et environ **1 000 images par classe** dans l'ensemble de test.

MNIST constitue un choix idéal pour ce type d'étude , il est suffisamment simple pour permettre des entraînements rapides sur de nombreuses configurations, bien connu et référencé dans la littérature, et suffisamment riche pour mettre en évidence des différences significatives entre stratégies de défense. Sa simplicité garantit que les écarts de performances observés entre agrégateurs reflètent bien l'effet des attaques, et non une difficulté intrinsèque de la tâche de classification.

Un jeu de données plus complexe, comme **CIFAR-10** [11], composé de 60 000 images couleur RGB 32×32 en 10 classes, serait davantage adapté à un réseau de neurones convolutionnel (CNN). Les CNN exploitent la structure spatiale des images grâce à des couches de convolution, de pooling et d'activation, et obtiennent des performances nettement supérieures sur des tâches visuelles complexes. Dans une suite à ce travail, CIFAR-10 associé à un CNN constituerait une étape naturelle pour tester la robustesse des agrégateurs dans un contexte complexifié.

3.2. Distributions IID et Non-IID

La manière dont les données sont réparties entre les clients a une influence déterminante sur la difficulté du problème.

En configuration **IID** (*Independent and Identically Distributed*), chaque client reçoit un échantillon aléatoire et uniformément distribué de l'ensemble des données. Chaque classe est ainsi représentée de manière équilibrée chez chaque client. Les gradients honnêtes produits sont donc similaires entre clients et pointent naturellement dans la même direction. Dans ce contexte, détecter une mise à jour byzantine est relativement accessible , elle se démarque du consensus.

En configuration **Non-IID**, la répartition est biaisée , chaque client ne possède que certaines classes, ou les observe avec des fréquences très inégales. Par exemple, un client peut n'avoir reçu que des images du chiffre 3 et du chiffre 7, tandis qu'un autre client aura exclusivement des 0 et des 1. Ce biais de distribution crée une hétérogénéité profonde entre clients , leurs gradients honnêtes divergent naturellement, même sans aucune attaque. Cette dispersion légitime complique drastiquement le travail des agrégateurs robustes, qui doivent contourner les perturbations introduites par les byzantins en faisant face à une variabilité naturelle due au Non-IID

4. ATTAQUES ET AGRÉGATEURS

4.1. Attaques byzantines étudiées

Les huit attaques non ciblées étudiées visent toutes à dégrader le modèle global, mais par des mécanismes très différents.

Sign Flipping (SF) [6] : l'attaquant inverse simplement le signe de son gradient réel : $\tilde{g} = -g_i$. L'effet est brutal : le modèle est poussé systématiquement dans la mauvaise direction à chaque iteration. Simple mais redoutablement efficace contre les agrégateurs qui ne filtrent pas les valeurs extrêmes.

Inner Product Manipulation (IPM) [7] : plutôt que d'agir brutalement, l'attaquant envoie un gradient dont le produit scalaire avec la vraie moyenne des gradients honnêtes est fortement négatif. Cela dévie l'optimisation de manière ciblée et structurée, rendant l'attaque plus difficile à détecter.

Optimal IPM (Opt-IPM) : version optimisée d'IPM, qui maximise analytiquement l'angle entre le gradient malveillant et la moyenne honnête, tout en restant discret.

A Little Is Enough (ALIE) [8] : attaque particulièrement sournoise. L'attaquant analyse la distribution statistique des gradients honnêtes et injecte une perturbation calibrée pour se fondre dans leur bruit naturel. L'attaque est "invisible" aux yeux de nombreuses défenses , le gradient malveillant est statistiquement plausible, mais oriente subtilement l'apprentissage dans la mauvaise direction.

Optimal ALIE (Opt-ALIE) : variante d'ALIE qui optimise le vecteur d'attaque pour maximiser son impact tout en évitant d'être éliminé par les agrégateurs basés sur la norme ou la distance.

Mimic : l'attaquant copie exactement le gradient d'un client honnête. En se déguisant en participant légitime, il trompe les défenses basées sur la similarité qui ne peuvent plus le différencier d'un client normal. Cette attaque révèle une faiblesse profonde des méthodes de sélection par proximité.

Gaussian : le gradient malveillant est tiré d'une distribution gaussienne centrée sur zéro, independante de toute donnée réelle. L'attaque est brute et non structurée , elle injecte un bruit aléatoire. Si elle semble naïve, elle se révèle dévastatrice pour les agrégateurs sans mécanisme de filtrage, et ses effets sont amplifiés par le contexte Non-IID.

Infinity : les gradients malveillants ont une norme extrêmement élevée, visant à dominer la somme agrégée et à écraser les contributions honnêtes.

4.2. Agrégateurs comparés

Douze agrégateurs sont évalués, des plus simples aux plus sophistiqués.

Average : moyenne arithmétique naïve de tous les gradients. Aucune résistance aux attaques . Un seul byzantin suffit à dévier l'ensemble de l'apprentissage. Sert de référence basse.

Median [9] : médiane coordonnée par coordonnée. Plus robuste qu'Average, car les valeurs extrêmes ont moins d'influence, mais vulnérable aux attaques qui ciblent la médiane elle-même.

TrMean [9] : moyenne tronquée. Élimine les f gradients les plus grands et les plus petits sur chaque coordonnée avant de faire la moyenne. Sa robustesse dépend du nombre de valeurs extrêmes supprimées avant le calcul de la moyenne.

Geometric Median [4] : point de l'espace qui minimise la somme des distances euclidiennes à l'ensemble des gradients. Très robuste aux vecteurs éloignés de la majorité, mais coûteux à calculer.

Krum [3] : sélectionne le gradient dont la somme des distances aux $n-f-2$ plus proches voisins est minimale. Il identifie ainsi le gradient le plus représentatif de la majorité. Très efficace contre les attaques directes, mais mis en défaut par Mimic.

Multi-Krum : variante de Krum sélectionnant les $m > 1$ meilleurs gradients au sens du score Krum, puis en faisant leur moyenne. Offre un meilleur compromis biais-variance que Krum seul.

Centered Clipping (CC) [5] : projette chaque gradient dans une boule de rayon τ centrée autour d'une estimation du centre honnête. Très efficace en Non-IID, car il adapte son centre aux données hétérogènes.

MDA (*Minimum Distance Aggregation*) : sélectionne le sous-ensemble de gradients dont la dispersion interne est minimale, en postulant que les honnêtes sont plus concentrés que les byzantins.

MoNNA : agrégateur basé sur le momentum , il utilise l'historique des mises à jour pour détecter les gradients inconsistants avec la dynamique observée.

Meamed : variante de la médiane qui utilise une estimation de la moyenne pour donner plus ou moins d'importance aux gradients, afin de combiner robustesse et efficacité.

CAF : filtre les gradients par blocs de coordonnées, cherchant à isoler les dimensions les plus corrompues.

SMEA : méthode récente qui sélectionne les gradients jugés fiables avant de calculer leur moyenne, afin de limiter l'influence des clients byzantins, montrant de très bonnes performances sur les attaques gaussiennes et Mimic en Non-IID.

5. PROTOCOLE EXPÉRIMENTAL

Les expériences ont été réalisées avec **5 clients honnêtes** et **2 clients byzantins**, sur le jeu de données MNIST, avec plusieurs iterations d'entraînement (jusqu'à 500 itérations pour les expériences longues sur Google Colab avec GPU, que nous avons réduites à environ 200 vers la fin par manque de temps et de ressources). Chaque scénario est défini dans un fichier **JSON** spécifiant l'attaque, l'agrégateur, la distribution (IID ou Non-IID) et le nombre d'iterations. Un moteur Python automatise le lancement et sauvegarde les résultats en CSV pour visualisation. Certains scénarios ont été répétés sur plusieurs seeds aléatoires (5 runs) pour stabiliser les résultats.

6. RÉSULTATS EXPÉRIMENTAUX

Parmi l'ensemble des expériences menées, nous avons cherché à identifier la meilleure attaque et le meilleur agrégateur selon un critère de pire cas. Une attaque est considérée comme la meilleure si elle arrive à minimiser la précision maximale de l'ensemble des agrégateurs. À l'inverse, le meilleur agrégateur est celui qui maximise la précision minimale face à toutes les attaques confondues.

6.1. La meilleure attaque et le meilleur agrégateur

L'analyse complète de toutes les courbes d'apprentissage confirme que l'attaque Optimal Inner Product Manipulation en distribution Non-IID s'impose définitivement comme la plus redoutable. Elle parvient à faire chuter drastiquement les performances d'agrégateurs pourtant très robustes en temps normal. Des méthodes comme SMEA ou Multi-Krum peinent à dépasser les 50 pour cent de précision sous cette attaque. Les expériences supplémentaires montrent que des attaques alternatives comme Gaussian ou A Little Is Enough sont finalement bien gérées par la majorité des défenses. L'attaque Gaussian injecte un bruit trop évident qui se retrouve très vite bloqué par les algorithmes. L'attaque A Little Is Enough est plus sournoise mais elle ne réussit pas à détruire le modèle d'apprentissage violement. En effet, il n'y a que l'aggrégateur Meamed qui peine à contrer cette attaque en distribution non-iid. Le biais naturel des données Non-IID reste le facteur aggravant qui complique la tâche des défenses et permet aux meilleures attaques de faire un maximum de dégâts.

Du côté des défenses, la moyenne tronquée (*TrMean*) et la médiane géométrique (*Geometric Median*) s'en sortent le mieux de façon globale. Même lorsque les expériences sont prolongées jusqu'à 500 itérations pour tester la résistance sur le long terme, ces méthodes prouvent leur grande fiabilité. Elles ne finissent pas forcément premières sur chaque graphique individuel, mais elles ne s'effondrent jamais. Elles assurent ainsi une performance minimale très robuste sur toutes les expériences confondues, là où d'autres agrégateurs tombent parfois à des scores proches du hasard sur des attaques spécifiques. Il convient cependant de noter que la sévérité de l'attaque *Optimal IPM* parvient à affaiblir significativement les capacités de ces deux défenses résilientes, alors que des agrégateurs plus sophistiqués, tels que *MoNNA*, *MDA* et *CAF*, ont su démontrer leur efficacité dans ce contexte.

6.2. Expérience 1 avec IPM en distribution IID

L'attaque Inner Product Manipulation (IPM) reste assez simple dans son approche algorithmique. L'attaquant construit un gradient dont le produit scalaire avec la direction honnête est négatif pour dévier le modèle. L'effet est subtil mais profond. Plutôt que de détruire brutalement les mises à jour, l'attaque dévie progressivement le modèle dans une mauvaise direction.

Dans cette configuration IID, l'agrégateur Multi-Krum atteint la meilleure précision finale avec 0.9672. Ce résultat est remarquable, mais il est particulièrement intéressant de noter que l'agrégateur naïf Average résiste étonnamment bien à cette attaque spécifique en IID, parvenant à maintenir près de 95% de précision (0.9491). L'agrégateur Centered Clipping offre d'ailleurs des performances similaires (0.9491). Le gain de Multi-Krum reste néanmoins significatif car il réussit à optimiser l'entraînement au maximum, écartant avec plus de précision les vecteurs byzantins pour frôler les 97%.

Ce succès s'explique par la répartition des données. En

IID, les gradients honnêtes sont très similaires entre eux. Multi-Krum calcule les distances entre les vecteurs et identifie naturellement le groupe majoritaire. Il écarte facilement les byzantins car ils s'éloignent trop du consensus.

6.3. Expérience 2 avec Gaussian en distribution Non-IID

L'attaque Gaussian injecte simplement du bruit aléatoire. C'est l'attaque la plus naïve possible.

En distribution Non-IID, l'agrégateur SMEA obtient la meilleure performance avec une précision de 0.9695. En distribution IID sur la même attaque Gaussian, SMEA reste d'ailleurs en tête (0.9717), talonné de très près par Multi-Krum (0.9715) et Geometric Median (0.971). Le contraste le plus frappant concerne la moyenne classique (Average), alors qu'elle parvenait à maintenir environ 95 pour cent face à IPM, elle s'effondre complètement sous l'attaque Gaussian, tombant à 22.3% en IID et à environ 10% en Non-IID (soit l'équivalent d'une classification totalement au hasard).

Le point le plus intéressant est de voir à quel point la robustesse dépend du contexte. Par exemple, Multi-Krum brille face aux attaques en IID, mais son efficacité chute drastiquement face à IPM en Non-IID (tombant à moins de 59 % de précision). La distribution Non-IID amplifie dramatiquement la difficulté de l'apprentissage. Les gradients honnêtes sont déjà très dispersés à cause de la répartition inégale des données. Les agrégateurs ont alors énormément de mal à séparer la variabilité légitime des perturbations malveillantes.

Table 1: Accuracy finale comparée sur IPM et Gaussian.

Agrégateur	**IPM (IID)**	**IPM (Non-IID)**	**Gaussian (IID)**	**Gaussian (Non-IID)**
Average	0.94912	0.9473	0.223	∼0.10
Geometric Median	0.9636	0.9375	0.971	0.967
Multi-Krum	**0.9672**	0.5895	0.9715	0.9644
CC	0.9491	0.9472	0.9518	0.94008
SMEA	0.9501	0.4609	**0.9717**	**0.9695**

6.4. Absence d'agrégateur universel

Le constat le plus saillant de ce travail est qu'aucun agrégateur ne domine toutes les situations. Multi Krum excelle par exemple face à l'attaque IPM en distribution IID mais reste très faible dans un contexte Non IID. Les deux graphiques placés juste en dessous permettent de comparer visuellement cette chute de performance lors du passage d'une distribution à l'autre.

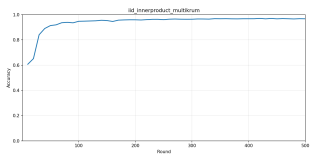


Figure 1: Multi Krum avec IPM(IID)

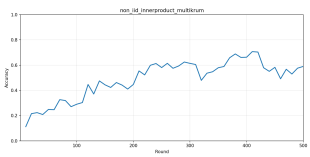


Figure 2: Multi Krum avec IPM(Non IID)

SMEA est optimal face à Gaussian en Non-IID mais se révèle très fragile sur d'autres attaques tel que *IPM*. Le choix d'un agrégateur doit impérativement dépendre d'une analyse du profil de menace prévu dans le contexte de déploiement.

6.5. Impact du cadre Non-IID

La distribution Non-IID représente systématiquement le scénario le plus exigeant. En IID, les gradients honnêtes forment un groupe compact ce qui rend la détection des byzantins plus évidente. En Non-IID, ce groupe s'étend. Les agrégateurs basés sur la concentration comme Krum ou Multi-Krum éliminent parfois des gradients honnêtes légitimement atypiques, ce qui dégrade leurs propres capacités d'apprentissage.

6.6. Limites du travail

Toutes les attaques n'ont pas pu être évaluées avec la même profondeur. En particulier, l'attaque *Optimal ALIE* a dû être écartée des expériences longues, son temps d'exécution démesuré provoquant des échecs systématiques sur Google Colab. Cette lourdeur s'explique par la mécanique de l'attaque, qui à chaque itération, doit résoudre un coûteux problème d'optimisation sous contraintes pour calibrer parfaitement sa perturbation sur la distribution des gradients honnêtes. Par ailleurs, nos résultats reposent sur MNIST, un jeu de données classique mais simple. Des expériences futures sur CIFAR-10 avec un réseau convolutif pourraient affiner ces conclusions. Enfin, les paramètres des agrégateurs ont été utilisés avec leurs valeurs par défaut, sans réglage spécifique par attaque.

7. CONCLUSION

Ce TER a permis de mettre en place un pipeline expérimental automatisé pour comparer douze agrégateurs robustes face à huit attaques byzantines non ciblées. Ces simulations ont été réalisées sur MNIST avec des distributions IID et Non-IID.

L'intérêt de l'apprentissage fédéré réside dans sa capacité à exploiter des données distribuées sans les centraliser. Sa vulnérabilité aux clients byzantins est bien réelle. La moyenne classique tombe à 10 pour cent de précision sous de nombreuses attaques, là où des défenses adaptées maintiennent d'excellents scores. Le choix de l'agrégateur est déterminant et aucune méthode n'est universellement parfaite. L'hétérogénéité des données en Non-IID rend toujours le problème bien plus difficile à résoudre.

La bibliothèque ByzFL constitue un outil solide pour poursuivre ces investigations. Il sera possible d'ajouter de nouvelles configurations d'attaques ou d'utiliser des modèles d'intelligence artificielle plus complexes pour prolonger ce travail de recherche.

8. BILAN PERSONNEL

Bilan de Samy

Ce projet a été pour moi une découverte concrète de ce que signifie mener une étude expérimentale scientifique. J'avais des notions théoriques sur l'apprentissage automatique, mais l'expérience de construire un pipeline complet, de la définition des scénarios JSON jusqu'à la génération des courbes d'analyse, m'a apporté une vision bien plus globale du processus de recherche. J'ai particulièrement apprécié le travail d'analyse des résultats, qui m'a obligé à réfléchir au-delà des chiffres et à formuler des explications formelle. Ce TER m'a aussi enseigné la valeur de l'organisation : structurer un dépôt, nommer proprement les

fichiers de résultats, répéter les expériences sur plusieurs seeds, autant d'habitudes que je considère désormais indispensables dans tout travail scientifique.

Bilan de Célia

Ce TER a été l'occasion pour moi d'aborder pour la première fois la problématique de la sécurité en apprentissage automatique, et plus précisément en apprentissage fédéré. Avant ce projet, je n'avais qu'une connaissance superficielle de ce domaine. Travailler sur les attaques byzantines et les mécanismes de défense m'a permis de comprendre que la robustesse est un problème fondamentalement contextuel, autrement dit, il n'existe pas de solution miracle, et chaque agrégateur a ses forces et ses failles selon le scénario. J'ai également développé des compétences pratiques en automatisation d'expériences et en analyse de courbes. La difficulté principale a été de gérer la quantité de configurations à tester et d'en tirer des conclusions claires et justifiées. Ce travail m'a donné envie d'aller plus loin sur les questions de confidentialité et de sécurité des systèmes d'IA distribués.

9. RÉFÉRENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, et B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[2] L. Lamport, R. Shostak, et M. Pease, "The Byzantine generals problem," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, 1982.

[3] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, et J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.

[4] Y. Chen, L. Su, et J. Xu, "Distributed statistical machine learning in adversarial settings: Byzantine gradient descent," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 2, 2017.

[5] S. P. Karimireddy, L. He, et M. Jaggi, "Byzantine-robust learning on heterogeneous datasets via bucketing," in *Proc. ICLR*, 2021.

[6] L. Li, W. Xu, T. Chen, G. B. Giannakis, et Q. Ling, "RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets," in *Proc. AAAI*, 2019.

[7] C. Xie, O. Koyejo, et I. Gupta, "Fall of empires: Breaking byzantine-tolerant SGD by inner product manipulation," in *Proc. UAI*, 2020.

[8] G. Baruch, M. Baruch, et Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Proc. NeurIPS*, 2019.

[9] D. Yin, Y. Chen, R. Kannan, et P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. ICML*, 2018.

[10] Y. LeCun, L. Bottou, Y. Bengio, et P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[11] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., Univ. Toronto, 2009.

[12] El Mhamdi *et al.*, "ByzFL: A Python library for Byzantine-robust federated learning simulation," `https://github.com/LPD-EPFL/ByzFL`, 2023.